

# The perror() Function ⚠

---

The perror() function is used to show error description. It displays string you pass to it, followed by a colon, a space, and then textual representation of current errno value.



## Syntax

```
void perror(const char *str);
```



## Parameters

**str:** It is a string containing a custom message to be printed before the error message itself.



## Example

```
// C implementation to see how perror() function is used to
// print error messages.
#include <errno.h>
#include <stdio.h>
#include <string.h>

int main()
{
    FILE* fp;
    // If a file is opened which does not exist,
    // then it will be an error and corresponding
    // errno value will be set
    fp = fopen(" Error Handling.txt ", "r");
    // opening a file which does
    // not exist.
    printf("Value of errno: %d\n ", errno);
    perror("Message from perror");
    return 0;
}
```

## Output:

```
Value of errno: 2
```

## The strerror() Function

The strerror() function is also used to show error description. This function returns a pointer to textual representation of current errno value.



### Syntax

```
char *strerror(int errnum);
```



## Parameters

**errno:** It is error number (errno).



## Example

```
// C implementation to see how strerror() function is used
// to print error messages.
#include <errno.h>
#include <stdio.h>
#include <string.h>

int main()
{
    FILE* fp;
    // If a file is opened which does not exist,
    // then it will be an error and corresponding
    // errno value will be set
    fp = fopen(" Error Handling.txt ", "r");
    // opening a file which does
    // not exist.
    printf("Value of errno: %d\n", errno);
    printf("The error message is : %s\n", strerror(errno));
    return 0;
}
```

## Output:

```
Value of errno: 2  
The error message is: No such file or directory
```

2/8

## Uses of ferror() Function

The ferror() function is used to check whether an error occurred during a file operation.



## Syntax

```
int ferror(FILE *stream);
```



## Parameters

**stream:** It is a pointer that points to FILE for which we want to check error.



## Return Value

It returns a non-zero value if an error occurred, otherwise it returns 0.



## Example

```
// C program to demonstrate ferror() function
#include <stdio.h>

int main()
{
    // Open file in read mode
    FILE* file = fopen("nonexistent_file.txt", "r");
    if (file == NULL)
    {
        // Print an error message
        // if file opening fails
        perror("Error opening file");
        // Return with non-zero exit status to
        // indicate an error
        return 1;
    }
    int c;
    // Process character
    // Add your code here to perform operations on each
    // character read from file
    while ((c = fgetc(file)) != EOF) {
    }
    if (ferror(file)) {
        // Print an error message if an error occurred
        // during file reading
        printf(
            "An error occurred while reading file.\n");
    }
    else {
        // Print success message if file reading completed
        // without errors
        printf("File read successfully.\n");
    }
    // Close file
    fclose(file);
    // Return with zero exit status to indicate successful
    // execution
    return 0;
}
```

**Output:**

Error opening file: No such file or directory

3/8

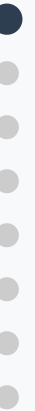
## 🖌️ The Uses of `clearerr()` Function

The `clearerr()` function is used to clear both end-of-file and error indicators for a file stream.



### Syntax

```
void clearerr(FILE *stream);
```







## Parameters

**stream:** It is a pointer that points to FILE for which we want to check error.



## Example

```
#include <stdio.h>

int main()
{
    FILE* file = fopen("file.txt", "r");
    // Open file in read mode
    if (file == NULL)
    {
        // Print an error message
        // if file opening fails
        perror("Error opening file");
        // Return with non-zero exit status to
        // indicate an error
        return 1;
    }
    // Perform file operations
    // Add your code here to perform operations on the
    // opened file
    if (ferror(file)) {
        // Print an error message
        // if an error occurred
        // during file operations
        printf("An error occurred while performing file "
```

```
    "operations.\n");  
}  
// Clear error indicators for the  
// file stream  
clearerr(file);  
// Continue with file operations  
// Add your code here to continue with further  
// operations on the file  
// Close file  
fclose(file);  
return 0;  
}
```

## ⚠ Divide by Zero Errors Function

A common pitfall made by C programmers is not checking if a divisor is zero before a division command. Division by zero leads to undefined behavior, there is no C language construct that can do anything about it. Your best bet is to not divide by zero in the first place, by checking the denominator.



## Example

```
// C program to check and rectify
// divide by zero condition
#include <stdio.h>
#include <stdlib.h>
void function(int);
int main()
{
    int x = 0;
    function(x);
    return 0;
}
void function(int x)
{
    float fx;
    if (x == 0) {
        printf("Division by Zero is not allowed");
        fprintf(stderr, "Division by zero! Exiting...\n");
        exit(EXIT_FAILURE);
    }
    else
    {
        fx = 10 / x;
        printf("f(x) is: %.5f", fx);
    }
}
```

## Output:

```
Division by zero! Exiting ....
```



# ■ Program Exit Status

Exit status is the value returned by the program after its execution is completed which tells the status of the execution of the program.

The C standard specifies two constants: `EXIT_SUCCESS` and `EXIT_FAILURE`, that may be passed to `exit()` to indicate successful or unsuccessful termination, respectively. These are macros defined in header file.



## Example

```
// C implementation which shows
// use of EXIT_SUCCESS and EXIT_FAILURE.
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
```

```
#include <string.h>

int main()
{
    FILE* fp;
    fp = fopen("filedoesnotexist.txt", "rb");
    if (fp == NULL)
    {
        printf("Value of errno: %d\n", errno);
        printf("Error opening the file: %s\n",
            strerror(errno));
        perror("Error printed by perror");
        exit(EXIT_FAILURE);
        printf("I will not be printed\n");
    }
    else
    {
        fclose(fp);
        exit(EXIT_SUCCESS);
        printf("I will not be printed\n");
    }
    return 0;
}
```

## Output:

```
Value of errno: 2
Error opening the file: No such file or directory
Error printed by perror: No such file or directory
```

# Error Handling Patterns

This chapter showed you how to perform error handling in C. Function Split tells you to split your functions into smaller parts to make error handling of these parts easier. A Guard Clause for your functions checks pre-conditions of your function and returns immediately if they are not met. This leaves fewer error-handling obligations for the rest of that function.

Instead of returning from the function, you could also abort the program, adhering to the Samurai Principle. When it comes to more complex error handling – particularly in combination with acquiring and releasing resources – you have several options.



## Goto Error Handling

Makes it possible to jump forward in your function to an error-handling section.



## Cleanup Record

Stores info about which resources require cleanup, and performs it by the end of the function.



## Object-Based Error Handling

Uses separate initialization and cleanup functions



similar to constructors and destructors.

7/8

## Conclusion

With these error-handling patterns in your repertoire, you now have the skill to write small programs that handle error situations in a way that ensures code stays maintainable.

### Key Takeaways

- ✓ **Function Split:** Break down complex functions into smaller, more manageable parts
- ✓ **Guard Clause:** Check pre-conditions early and return if not met
- ✓ **Samurai Principle:** Abort when errors cannot be handled
- ✓ **Goto Error Handling:** Jump to cleanup code when errors occur
- ✓ **Cleanup Record:** Track resources that need cleanup



## **perror()**

Prints error messages with custom text



## **strerror()**

Returns string description of error codes



## **ferror()**

Checks for file operation errors



## **clearerr()**

Clears error indicators for file streams

